Programs for rank augmentation and its obstruction

Human Compression project

This appendix prints all three complete programs in the rank augmentation application. It replaces the earlier appendix, which printed only the two GAP files. No mathematical program has changed. The source-example computations, bounded validation and certificate for the eighteen-point obstruction retain their separate roles. The last certificate supports a complete written proof; the validation does not prove a statement for unbounded degree.

The five-page common paper, three-page source application and three-page obstruction proof remain the complete written boundary. Their costs are paid in addition to whichever complete program representation is chosen. The operational drivers and the two partial Lean files are inventoried separately. This appendix adds no mathematical result or source credit.

The three source examples

```
# Complete three source examples, with actual parabolic and orbit computations.
SizeScreen([240,60]);;
Need:=function(x,m)if not x then Error(m);fi;end;;
Par:=function(xs,I)return Group(Concatenation([()],xs{I}));end;;
AllIP:=function(xs)local ss,i,j,n;ss:=Combinations([1..Length(xs)]);n:=0;
 for i in ss do for j in ss do
  Need(Intersection(Par(xs,i),Par(xs,j))=Par(xs,Intersection(i,j)),"old intersection property");n:=n+1;
 od;od;return n;end;;
cases:=[
 rec(n:=9,a:=(2,3)(6,9)(7,8),q:=(1,2)(3,4)(5,6),c:=(4,5)(6,7)(8,9),t:=(1,2),big:=[2..9],primitive:=true),
 rec(n:=10,a:=(1,2)(3,4)(9,10),q:=(2,3)(4,5)(6,7)(8,9),c:=(5,6)(7,8),t:=(2,3),big:=[3..10],primitive:=false),
 rec(n:=13,a:=(1,2)(3,4)(7,9)(8,10),q:=(2,3)(4,5)(6,7)(10,11)(12,13),c:=(5,6)(7,8)(9,10)(11,12),t:=(2,3),big:=[3..13],primitive:=true)
];;
rows:=[];;
for e in cases do
 old:=[e.a,e.q,e.c];;b:=e.t*e.q;;new:=[e.t,e.a,b,e.c];;
 Need(Size(Group(old))=Factorial(e.n),"original symmetric group");
 Need(ForAll(old,g->Order(g)=2) and e.a*e.c=e.c*e.a,"original string");
 count:=AllIP(old);;B:=Group([e.a,b]);;H:=Group([e.t,e.a,b]);;K:=Group([e.a,b,e.c]);;
 Need(ForAll(Orbits(B,[1..e.n]),o->Length(o)<=3),"middle orbit bound");
 Need(IsPrimitive(Action(K,e.big))=e.primitive,"specified primitive action");
 if e.n<>10 then Need(Size(Action(K,e.big))=Factorial(Length(e.big)),"full symmetric projection");fi;
 Need(Intersection(H,K)<>B,"new intersection failure");
 if e.n=9 then Need((2,3,4) in Intersection(H,K) and not (2,3,4) in B,"source first independent triple witness");fi;
 if e.n=10 then
  Need(Set(Orbit(K,[3,10],OnSets))=Set([[3,10],[4,9],[5,8],[6,7]]),"literal source block system");
  Need((1,2) in Intersection(H,K) and not (1,2) in B,"source pair-flip witness");
 fi;
 if e.n=13 then
  Need(ForAll([e.a,b,e.c],g->SignPerm(g)=1),"all three even marks");
  Need((3,4,5) in Intersection(H,K) and not (3,4,5) in B,"source third independent triple witness");
 fi;
 Add(rows,[e.n,Size(B),Size(H),Size(K),Size(Intersection(H,K)),List([e.a,b,e.c],SignPerm),count]);
od;
Need(rows=[[9,12,288,40320,72,[-1,1,-1],64],[10,12,1440,768,24,[-1,-1,1],64],[13,12,1440,39916800,72,[1,1,1],64]],"complete fixed source-example
table");
Print("SOURCE_EXAMPLES ",rows,"\n");
Print("PASS three source examples; old_IP_pairs=192; all output defects witnessed\n");
QUIT_GAP(0);
```

Bounded validation of the common argument

```
# Exhaustive finite diagnostics within the explicitly bounded template space.
# These samples are not premises of any quantified written theorem.
SizeScreen([240,60]);;
Need:=function(x,m)if not x then Error(m);fi;end;;
Par:=function(xs,I)return Group(Concatenation([()],xs{I}));end;;
IsC:=function(xs)local ss,i,j;ss:=Combinations([1..Length(xs)]);
 for i in ss do for j in ss do
  if Intersection(Par(xs,i),Par(xs,j))<>Par(xs,Intersection(i,j)) then return false;fi;
 od;od;return true;end;;
TailList:=function(m,n)local tails,k,nxt,s,a,b,u,z;
 tails:=[[[],[],m]];
 for k in [1..n-m] do
  nxt:=[];
  for s in tails do a:=s[1];b:=s[2];u:=s[3];
   if u=n then Add(nxt,s);else
    Add(nxt,[a,b,u+1]);
    if u+2<=n then z:=(u+1,u+2);
     Add(nxt,[Concatenation(a,[z]),b,u+2]);
     Add(nxt,[a,Concatenation(b,[z]),u+2]);
```

```
     Add(nxt,[Concatenation(a,[z]),Concatenation(b,[z]),u+2]);
    fi;
    if u+3<=n then Add(nxt,[Concatenation(a,[(u+1,u+2)]),Concatenation(b,[(u+2,u+3)]),u+3]);fi;
   fi;
  od;tails:=Set(nxt);
 od;return Filtered(tails,s->s[3]=n);
end;;
templates:=[rec(m:=4,a:=(2,3),b:=(3,4),t:=(1,2),ends:=[1,2],u:=2),
 rec(m:=5,a:=(1,2)(3,4),b:=(4,5),t:=(2,3),ends:=[2,3],u:=3),
 rec(m:=6,a:=(2,3)(4,5),b:=(1,2)(5,6),t:=(3,4),ends:=[3,4],u:=4)];;
counts:=List([1..3],i->[0,0,0,0]);;R:=[];;
for d in [1..3] do e:=templates[d];
 for n in [Maximum(5,e.m)..8] do
  for tail in TailList(e.m,n) do
   a:=e.a*Product(tail[1],());;b:=e.b*Product(tail[2],());;t:=e.t;;
   central:=Intersection(Centralizer(SymmetricGroup(n),a),Stabilizer(SymmetricGroup(n),e.ends,OnTuples));;
   for c in Filtered(Elements(central),x->Order(x)=2) do
    counts[d][1]:=counts[d][1]+1;
    if not IsTransitive(Group([t,a,b,c]),[1..n]) then continue;fi;
    counts[d][2]:=counts[d][2]+1;
    Need(Length(Set([t,a,b,c]))=4,"distinct output involutions");
    B:=Group([a,b]);;H:=Group([t,a,b]);;K:=Group([a,b,c]);;
    Need(IsC([t,a,b]) and IsC([a,b,c]),"both actual C ends");
    full:=IsC([t,a,b,c]);;
    if e.m<6 then
     aout:=PermList(List([1..n],i->i));;bout:=aout;;
     aa:=List([1..n],i->i);;bb:=ShallowCopy(aa);;
     for i in [e.m+1..n] do aa[i]:=i^a;bb[i]:=i^b;od;
     Bout:=Group([PermList(aa),PermList(bb)]);;
     Need(H=Group(Concatenation(GeneratorsOfGroup(SymmetricGroup([1..e.m])),GeneratorsOfGroup(Bout))),"actual supported product");
     R:=Orbit(K,e.u);;triples:=Filtered(Orbits(B,[1..n]),o->Length(o)=3 and o[1] in R);;
     blockhyp:=Length(triples)=1 or not IsPrimitive(Action(K,R));;
     sufficient:=blockhyp and (e.m=4 or ForAll([a,b,c],x->SignPerm(x)=1));;
     if sufficient then Need(full,"four/five point positive theorem");counts[d][3]:=counts[d][3]+1;fi;
    else
     N:=Group([(3,4),(2,5),(1,6)]);;
     Need(Size(N)=8 and H=Group(Concatenation(GeneratorsOfGroup(N),[a,b])) and Size(Intersection(N,B))=1,"matching semidirect product");
     Need(full=(Size(Intersection(N,K))=1),"seven-test equivalence");
     if not 4 in Orbit(K,3) then Need(full,"separated-triple sufficient condition");counts[d][3]:=counts[d][3]+1;fi;
    fi;
    if full then counts[d][4]:=counts[d][4]+1;fi;
   od;
  od;
 od;
od;
Print("COUNTS [scanned,transitive,sufficient,C] ",counts,"\n");
Print("PASS bounded common-theorem diagnostics\n");
QUIT_GAP(0);
```

Certificate for the eighteen-point obstruction

```
#!/usr/bin/env python3
"""An elementary tuple-permutation certificate; no S18 enumeration or algebra package."""
import json
from collections import deque
N=18
ONE=tuple(range(N))

def require(value,message):
    if not value: raise ValueError(message)
def cycles(*cc):
    p=list(ONE)
    for cyc in cc:
        for x,y in zip(cyc,cyc[1:]+cyc[:1]):p[x-1]=y-1
    require(sorted(p)==list(ONE),'permutation');return tuple(p)
def mul(p,q):return tuple(q[i] for i in p)
def inverse(p):
    r=[0]*N
    for i,j in enumerate(p):r[j]=i
    return tuple(r)
def power(p,n):
    r=ONE
    while n:
        if n&1:r=mul(r,p)
        p=mul(p,p);n//=2
    return r
def word(s,marks):
    r=ONE
    for x in s:r=mul(r,marks[x])
    return r
def conjugate(p,g):return mul(mul(inverse(g),p),g)
def orbit_partition(gens):
    todo=set(range(N));parts=[]
    while todo:
        seed=min(todo);seen={seed};queue=deque([seed])
        while queue:
            x=queue.popleft()
            for g in gens:
                y=g[x]
                if y not in seen:seen.add(y);queue.append(y)
        parts.append(sorted(i+1 for i in seen));todo-=seen
```

```
    return parts
def subgroup(gens):
    seen={ONE};queue=deque([ONE])
    while queue:
        p=queue.popleft()
        for q in gens:
            s=mul(p,q)
            if s not in seen:seen.add(s);queue.append(s)
        require(len(seen)<=180,'only bounded proper old subgroups may be enumerated')
    return seen
A=cycles((1,2),(5,6),(7,8),(11,12),(13,14),(17,18))
B=cycles((2,3),(4,5),(7,9),(10,11),(14,15),(16,17))
C=cycles((3,4),(5,7),(6,8),(9,10),(11,13),(12,14),(15,16))
T=cycles((1,18))
TREE=[('',(4,9)),('q',(5,7)),('c',(3,10)),('qa',(6,8)),('cq',(2,11)),
 ('cqa',(1,12)),('cqc',(2,13)),('cqaq',(12,18)),('cqac',(1,14)),
 ('cqcq',(3,13)),('cqaqa',(11,17)),('cqacq',(15,18)),('cqcqa',(3,14)),
 ('cqcqc',(4,11)),('cqaqaq',(10,16)),('cqcqcq',(5,10)),('cqcqcqa',(6,10))]

def verify(t=T,w_exponent=8,tree=TREE):
    a,b,c=A,B,C;q=mul(t,b);marks={'a':a,'b':b,'c':c,'q':q}
    old=[a,q,c];new=[t,a,b,c]
    for xs in [old,new]:
        require(len(set(xs))==len(xs),'distinct marks')
        require(all(x!=ONE and power(x,2)==ONE for x in xs),'nonidentity involutions')
        for i in range(len(xs)):
            for j in range(i+2,len(xs)):
                require(mul(xs[i],xs[j])==mul(xs[j],xs[i]),'string commutations')
    require(sum(q[i]!=i for i in range(N))==14,'old middle has seven transpositions')
    require(c[0]==0 and c[17]==17,'chosen edge avoids c-edges')
    seed=power(word('aqcq',marks),45)
    require(seed==cycles((4,9)),'generation seed word')
    edges=[]
    for w,pair in tree:
        require(conjugate(seed,word(w,marks))==cycles(pair),'conjugate tree edge '+w)
        edges.append(cycles(pair))
    require(len(tree)==17 and orbit_partition(edges)==[list(range(1,19))],'connected spanning tree certificate')
    # The graph-transposition theorem now proves old generation of the full S18.
    r=mul(a,q);s=mul(q,c)
    require(r==cycles((1,3,2,18,16,17),(4,5,6),(7,8,9),(10,11,12),(13,15,14)),'old left rotation')
    require(mul(b,c)==cycles((2,4,7,10,13,11,9,5,3),(6,8),(12,14,16,17,15)),'right rotation')
    require(power(r,6)==ONE and all(power(r,i)!=ONE for i in range(1,6)),'left order six')
    require(power(s,90)==ONE and all(power(s,i)!=ONE for i in range(1,90)),'right order ninety')
    require(power(mul(t,a),4)==ONE and power(mul(t,a),2)!=ONE,'new first adjacent order four')
    require(power(mul(a,b),3)==ONE and mul(a,b)!=ONE,'new middle adjacent order three')
    proper={mask:subgroup([old[i] for i in range(3) if mask&(1<<i)]) for mask in range(7)}
    require([len(proper[i]) for i in [3,6]]==[12,180],'dihedral end orders')
    pairs=0;whole_cases=0
    for i in range(8):
        for j in range(8):
            if i==7 or j==7:
                # A subgroup intersects the whole generated group in itself.
                require((i&j)==(j if i==7 else i),'whole-group subset identity');whole_cases+=1
            else:require(proper[i]&proper[j]==proper[i&j],'actual proper parabolic intersection')
            pairs+=1
    middle=orbit_partition([a,b]);end=orbit_partition([t,a,b])
    require(middle==[list(range(k,k+3)) for k in range(1,19,3)],'six middle triples')
    require(end==[[1,2,3,16,17,18],[4,5,6],[7,8,9],[10,11,12],[13,14,15]],'six-point end orbit')
    w=mul(mul(mul(mul(a,power(mul(b,c),2)),b),a),power(mul(c,b),w_exponent)),a)
    require(power(mul(b,c),45)==cycles((6,8)),'witness initial transposition')
    require(w[5]==0 and w[7]==17,'witness endpoint transport')
    require(conjugate(power(mul(b,c),45),w)==t and t!=ONE,'disjoint-label intersection witness')
    return dict(points=N,old_group='S18 by connected transposition certificate',generation_tree_edges=len(tree),
        old_type=[6,90],new_adjacent_orders=[4,3,90],old_parabolic_pairs=pairs,directly_enumerated_proper_pairs=pairs-whole_cases,
        whole_group_cases_by_containment=whole_cases,max_enumerated_group_order=max(map(len,proper.values())),
        middle_orbits=middle,end_orbits=end,witness='((bc)^45)^(a(bc)^2ba(cb)^8a)=t',
        all_source_case_three_hypotheses_checked=True,old_marking_C=True,new_marking_C=False,
        full_symmetric_group_enumerated=False,uses_GAP=False,
        permutations_one_based={k:[x+1 for x in p] for k,p in {'a':a,'b':b,'c':c,'t':t,'q':q}.items()})

def main():
    result=verify();controls=[]
    for name,kwargs in [('wrong_power',{'w_exponent':7}),('missing_tree_edge',{'tree':TREE[:-1]}),
                        ('wrong_split_transposition',{'t':cycles((1,17))})]:
        try:verify(**kwargs)
        except ValueError as e:controls.append({'name':name,'rejected_at':str(e)})
        else:raise RuntimeError('Bad certificate accepted '+name)
    result.update(status='PASS',rejected_controls=controls,uniform_source_recovery=False)
    print(json.dumps(result,indent=2))
if __name__=='__main__':main()
```